Transformer 입문

기초 개념부터 NumPy · PyTorch 직접 구현까지

ChatGPT의 심장을 처음부터 끝까지 내 손으로 만들어 봅니다

나는 | AI를 | 배운다 → [0.2, -1.1, 0.7, ...] 숫자 벡터

튜링랩 프로그래밍학원
TuringLab. Programming Academy

TuringLab.

OVERVIEW

강의 로드맵: 6교시 (3시간 × 2회)

회차	교시	주제	직접 만드는 것
1회차	1교시	글자를 숫자로: 토큰화·임베딩, 트랜스포머 구조 한눈에	Step 1 토큰·임베딩
	2교시	순서와 어텐션: 위치 인코딩, Q·K·V, softmax, 마스크	Step 2~3 위치 인코딩·어텐션
	3교시	멀티헤드, FFN, 잔차연결, LayerNorm, 블록 조립	Step 4 멀티헤드 어텐션
2회차	4교시	학습 원리: 손실, 역전파, 자동미분 엔진 만들기	Step 5 미니 Autograd
	5교시	NumPy만으로 GPT 조립·학습·글 생성	Step 6 완성된 트랜스포머
	6교시	PyTorch로 실전 버전 + 검증 + 실습 문제	PyTorch Step 1~6

매 교시: 50분 수업 + 10분 휴식 | 준비물: Python 3.9+, NumPy, (6교시) PyTorch

GOAL

이 강의가 끝나면 할 수 있는 일

1. **설명하기**
   토큰화 → 임베딩 → 어텐션 → 블록 → 출력의 흐름을 내 말로 설명한다

2. **계산하기**
   어텐션 가중치를 손으로 계산하고 shape을 추적한다

3. **만들기 (NumPy)**
   PyTorch 없이 자동미분 엔진부터 트랜스포머까지 직접 만든다

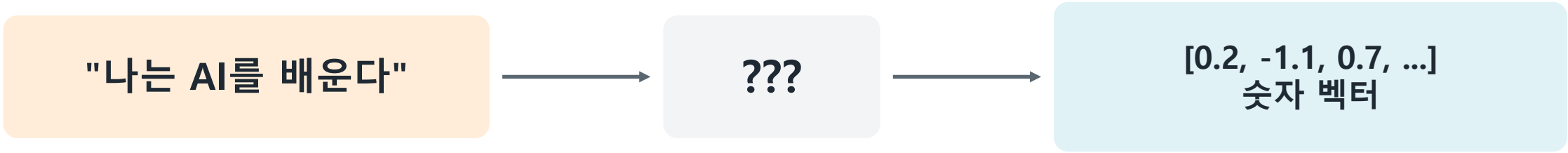
4. **만들기 (PyTorch)**
   같은 모델을 실전 방식으로 구현하고 내장 부품과 비교 검증한다

1교시

글자를 숫자로 바꾸기

토큰화 · 임베딩 · 트랜스포머는 왜 필요한가

컴퓨터는 글자를 모른다, 숫자만 안다



문제

신경망의 계산(곱하기·더하기)은
숫자에만 할 수 있다.

글자 "사과"를 어떻게 곱하고 더할까?

해결: 두 단계 변환

① 토큰화 글자 → 정수 번호
② 임베딩 번호 → 숫자 벡터

이 두 단계가 모든 언어모델의 출발점

① 토큰화(Tokenization): 문장을 조각내어 번호 붙이기

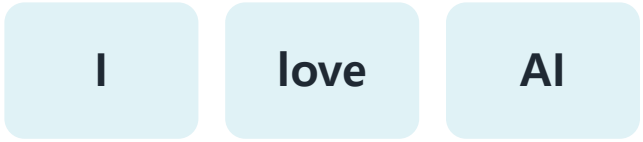
토큰(token) = 모델이 다루는 가장 작은 조각. 문자·단어·단어 조각(subword) 등 방식이 여러 가지다.

문자 단위



단순, 어휘가 작다 (이번 실습에서 사용)

단어 단위



직관적이지만 어휘가 매우 커진다

서브워드(BPE)



ChatGPT 등 실제 모델이 사용

핵심: 어떤 방식이든 "토큰 → 정수 번호"로 바뀐다. 번호는 단지 이름표일 뿐, 크기에 의미가 없다.

어휘집(vocab)과 인코딩 / 디코딩

텍스트 "hello"에서 쓰인 글자를 모아 정렬한 뒤, 순서대로 번호를 매긴다(사전순).

글자	e	h	l	o
번호	0	1	2	3

encode("hello")

h	e	l	l	o
↓				
1	0	2	2	3

stoi / itos 두 개의 사전

stoi : 글자 → 번호 {'e':0, 'h':1, ...}
itos : 번호 → 글자 {0:'e', 1:'h', ...}

학습할 때는 stoi, 글을 만들 때는 itos

vocab_size (어휘 크기)

서로 다른 토큰의 개수 = V
출력층이 "V개 중 다음 토큰은?"을 맞히게 된다.
(ChatGPT류는 보통 수만 ~ 수십만)

② 임베딩(Embedding): 번호를 의미 있는 숫자 벡터로

번호 3은 단지 이름표다. 그래서 토큰마다 "성격을 나타내는 숫자 묶음(벡터)"을 하나씩 준다.

임베딩 표 E (V × d_model)

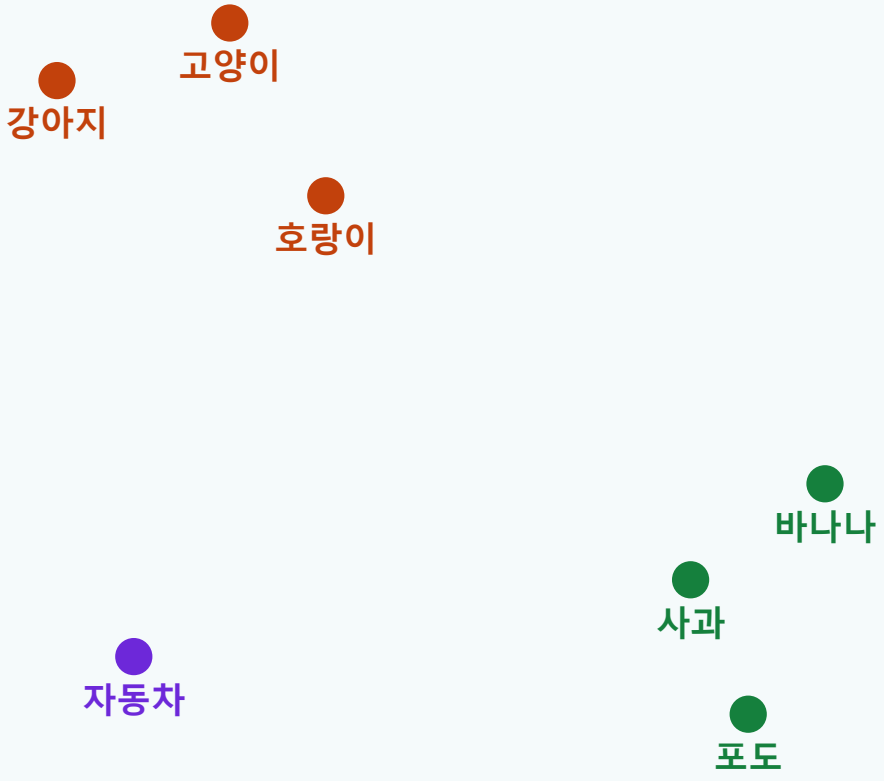
번호	d1	d2	d3	d4	
0	0.3	-1.2	0.8	0.1	
1	-0.5	0.4	0.2	1.1	
2	1.4	0.3	-0.7	0.6	← 2번 행
3	0.2	0.9	0.5	-0.4	

임베딩 조회 = E[번호]
처음엔 랜덤 값이지만 학습하며 의미가 담긴다.
d_model: 벡터 길이 (GPT-3는 12,288)

알아 두기

- 임베딩 표는 "학습되는 파라미터"다.
- "one-hot 벡터 × 표" 행렬곱과 같은 결과인데, 훨씬 빠른 조회 방식이다.
- 같은 토큰은 위치와 상관없이 처음엔 같은 벡터를 받는다. (위치 정보는 2교시에서 따로 더한다)
- 출력 shape: (T, d_model)
  T = 토큰 개수

임베딩 공간: 비슷한 의미는 가까이



(실제는 수백~수만 차원이라 그림은 2차원 단순화)

거리 = 의미의 차이

벡터가 가까울수록 의미가 비슷하다.
학습이 끝난 임베딩에서는
동물끼리, 과일끼리 모인다.

유명한 예: 단어 산술

왕 − 남자 + 여자 ≈ 여왕

벡터 계산으로 의미 관계가
표현되기도 한다.

[코드 Step 1] 토큰화와 임베딩 (NumPy)

numpy_version/step1_embedding.py

```
text = "hello transformer"
chars = sorted(set(text))          # 중복 제거 + 정렬
vocab_size = len(chars)            # 어휘 크기 V
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}

def encode(s): return [stoi[c] for c in s]
def decode(ids): return "".join(itos[i] for i in ids)

d_model = 8
rng = np.random.default_rng(0)   # 시드 고정 = 재현 가능
E = rng.normal(0, 1, (vocab_size, d_model))  # 임베딩 표

ids = np.array(encode("hello"))  # 정수 번호
x = E[ids]                       # 표에서 행 꺼내기
# x.shape = (5, 8) = (토큰 수 T, d_model)
```

코드 읽는 법

① set → sorted: 글자마다 항상 같은 번호가 나오도록 정렬한다.

② stoi / itos: 글자↔번호 변환용 사전 두 개.

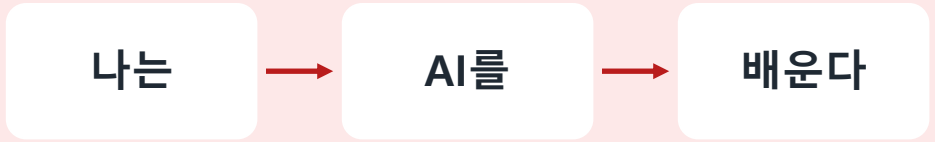
③ E는 (V, d_model) 크기의 학습 대상 표다.

④ E[ids]: 번호를 행 번호로 써서 벡터를 한꺼번에 꺼낸다.

⑤ 이 결과 x가 다음 단계(위치 인코딩, 어텐션)의 입력이 된다.

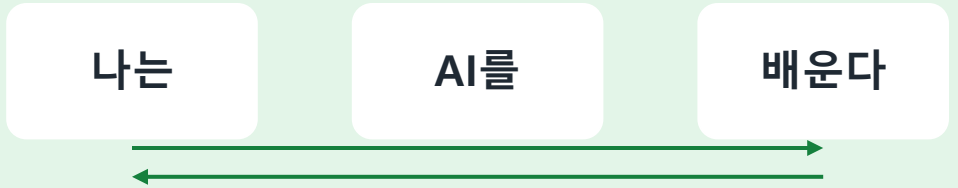
왜 트랜스포머인가? 기존 방식(RNN)의 한계

RNN: 한 단어씩 순서대로



- 앞 단어를 처리해야 다음 단어 가능 → 병렬화 어려움, 느림
- 문장이 길면 앞부분을 점점 잊어버림
- 멀리 떨어진 단어 사이 관계 파악이 어려움

Transformer: 모두를 한꺼번에



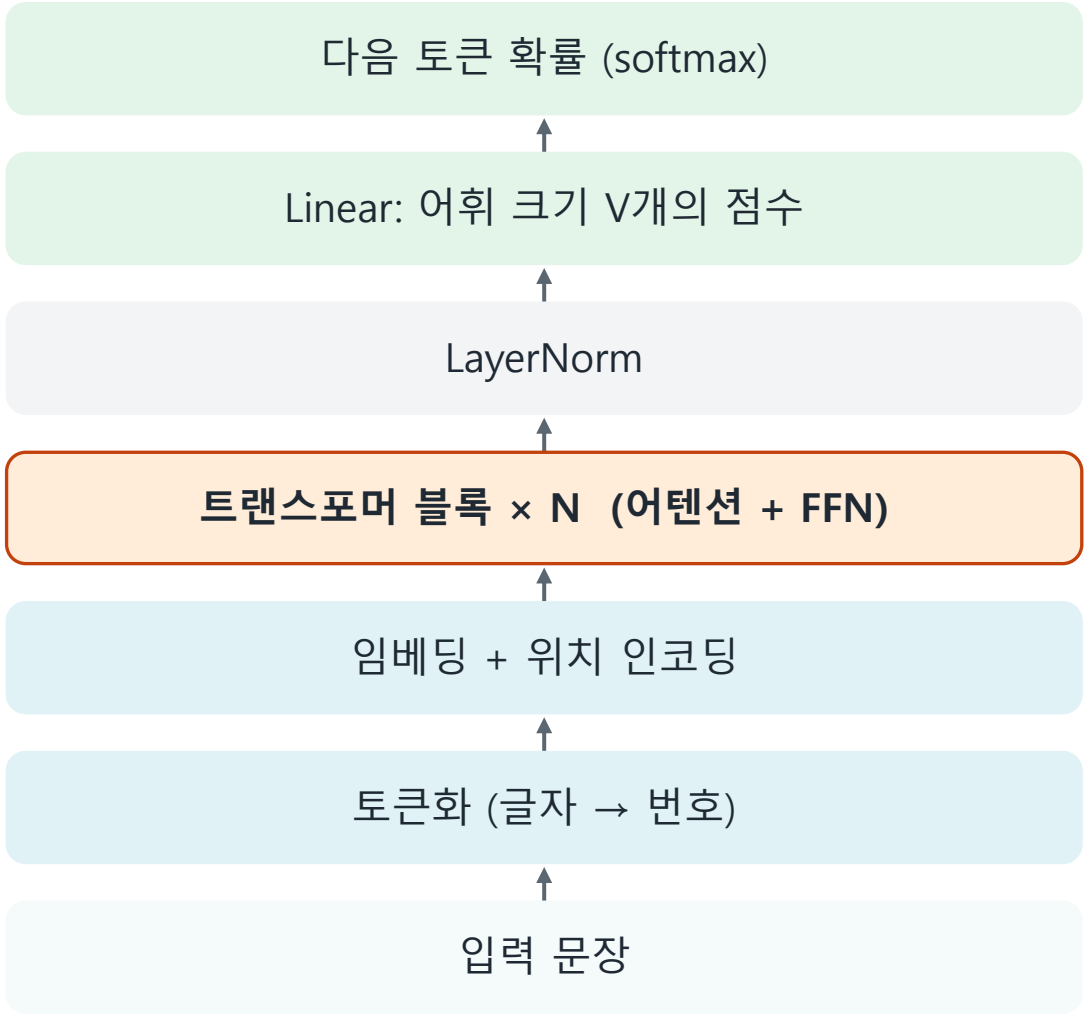
- 모든 단어가 모든 단어를 직접 참조(어텐션)
- 한꺼번에 계산 → GPU 병렬화에 최적, 빠름
- 거리와 상관없이 관계를 바로 연결

트랜스포머 연대기

2017	2018	2020	2022	2023~
Transformer 등장	**BERT · GPT-1**	**GPT-3**	**ChatGPT**	**멀티모달·에이전트**
논문 "Attention Is All You Need" (Google)	이해(BERT)와 생성(GPT) 두 갈래 시작	모델을 키울수록 능력이 크게 향상	대화형 서비스로 대중화	이미지·음성·코드·도구 사용으로 확장

오늘 만드는 구조 = GPT 계열이 쓰는 "디코더 트랜스포머"의 축소판. 원리는 ChatGPT와 같고 크기만 다르다.

트랜스포머 전체 구조 한눈에 (오늘의 지도)



블록 안에서 일어나는 일

x = x + 어텐션(LayerNorm(x))
x = x + FFN(LayerNorm(x))

아래에서 위로 한 방향

입력이 층을 통과할 때마다 단어 벡터가
문맥을 담아 점점 똑똑해진다.

마지막 한 줄

마지막 위치의 출력 점수 → softmax →
"다음에 올 토큰" 확률 분포

1교시 정리 퀴즈

Q1 컴퓨터에 글자를 넣기 전에 왜 숫자로 바꿔야 할까요?

Q2 어휘 크기가 28, d_model이 32라면 임베딩 표의 shape은?

Q3 RNN 대비 트랜스포머의 장점 두 가지는?

→ 5분 쉬고 2교시에서 "순서"와 "어텐션"을 배웁니다.

2교시

순서와 어텐션

위치 인코딩 · Query / Key / Value · softmax · 마스크

문제: 어텐션은 단어의 순서를 모른다

개가 사람을 물었다

사람이 개를 물었다

뜻이 완전히 다르다!

왜 문제일까?

어텐션은 단어 집합을 한꺼번에 처리한다.
단어가 같으면 순서를 바꿔도
같은 계산이 되어 버린다.
(RNN은 순서대로 읽어서 자동으로 알았다)

해결: 위치 표시를 더해 준다

각 토큰 벡터에 "나는 몇 번째"를
나타내는 위치 벡터를 더한다.

입력 = 단어 임베딩 + 위치 인코딩

위치 인코딩 아이디어: 시계의 바늘

시침·분침·초침은 서로 다른 속도로 돈다. 세 바늘의 각도 조합으로 하루 어느 시각이든 유일하게 표현할 수 있다.

빠르게 도는 파형

차원 0, 1
위치의 "세밀한" 차이

중간 속도 파형

차원 2, 3
위치의 "중간" 차이

느리게 도는 파형

차원 4, 5...
위치의 "큰" 차이

사인·코사인을 쓰는 이유

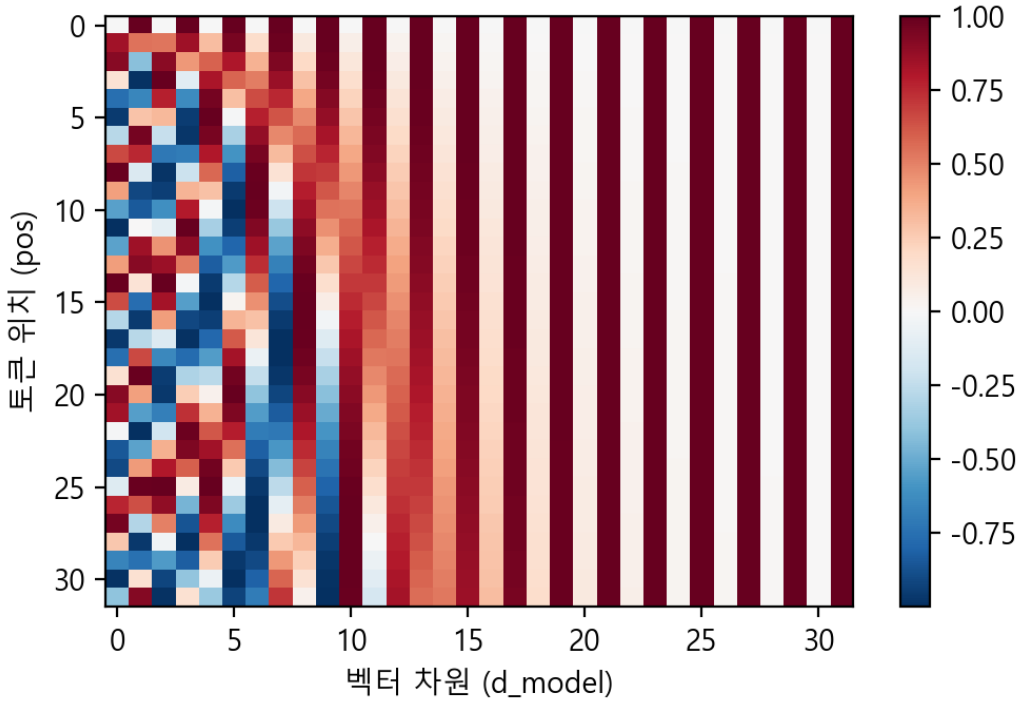
- 값이 항상 −1 ~ 1 사이라 임베딩을 망가뜨리지 않는다.
- 학습이 필요 없고, 훈련 때 못 본 긴 문장에도 계산할 수 있다.
- 가까운 위치끼리는 비슷한 벡터가 된다.

위치 인코딩 공식과 모양

공식 (pos = 위치, i = 차원 쌍 번호)

$$PE(pos, 2i) = sin\left(pos/10000^{(2i/d)}\right)$$
$$PE(pos, 2i+1) = cos\left(pos/10000^{(2i/d)}\right)$$

- 짝수 차원은 sin, 홀수 차원은 cos
- 분모 $10000^{(2i/d)}$: 차원이 커질수록 파형이 느려진다
- 임베딩에 더한다 (이어 붙이지 않는다)
- x_in = 임베딩 + PE[:T] → shape 그대로 (T, d)



가로: 차원 / 세로: 위치. 왼쪽 차원일수록 빠르게 변한다.

[코드 Step 2] 위치 인코딩 (NumPy)

numpy_version/step2_positional_encoding.py

```
def positional_encoding(max_len, d_model):
    pos = np.arange(max_len)[:, None]       # (max_len, 1)
    i = np.arange(0, d_model, 2)[None, :]   # (1, d/2)
    angle = pos / np.power(10000.0, i / d_model)
    pe = np.zeros((max_len, d_model))
    pe[:, 0::2] = np.sin(angle)             # 짝수 열
    pe[:, 1::2] = np.cos(angle)             # 홀수 열
    return pe

pe = positional_encoding(16, 8)
x_in = x + pe[:x.shape[0]]   # 임베딩에 '더하기'

# 가까운 위치일수록 내적이 크다 (검산)
pe = positional_encoding(64, 32)
pe[0] @ pe[1]    # 15.3  (거리 1)
pe[0] @ pe[48]   # 5.3   (거리 48)
```

코드 읽는 법

① [:, None] / [None, :] 로 축을 만들어 (위치 × 차원) 표를 한 번에 계산 (브로드캐스팅).

② 0::2 는 0,2,4… 번째 열, 1::2 는 1,3,5… 번째 열.

③ pe 표는 학습하지 않는 상수다. 한 번 만들어 계속 쓴다.

④ x + pe[:T] : 같은 shape이어서 원소별 덧셈이 가능하다.

⑤ 마지막 두 줄: 거리가 멀어질수록 내적이 줄어드는 것을 확인한다.

셀프 어텐션 직관: 도서관에서 책 찾기

Query (질문)

"AI 입문서 있나요?"

내가 지금 찾고 싶은 것

Key (색인·이름표)

책등에 적힌 제목·분류

"나는 이런 정보를 가졌어요"

Value (책의 내용)

실제로 가져올 내용

선택된 책에서 얻는 정보

질문(Q)과 이름표(K)가 잘 맞을수록 → 그 책의 내용(V)을 많이 가져온다

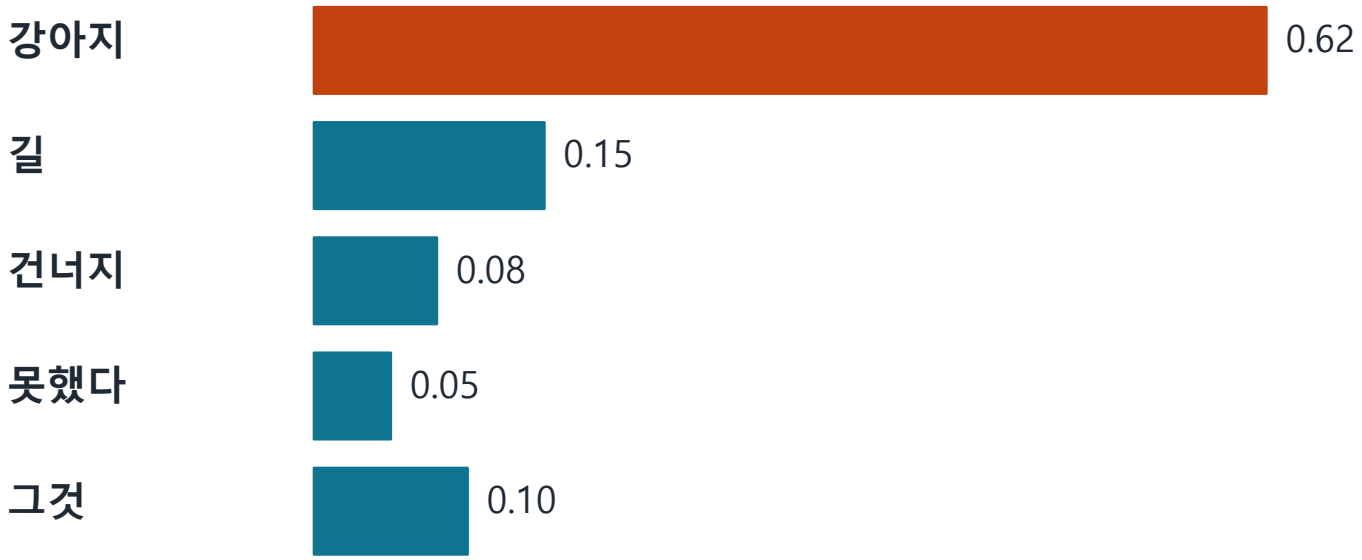
문장 속 모든 단어가 각자 Q, K, V를 가진다. 모든 단어가 "나는 누구를 얼마나 참고할까?"를 동시에 계산한다.

예: "그것"은 무엇을 가리킬까?

"강아지가 길을 건너지 못했다. 그것은 너무 피곤했다."

사람은 "그것 = 강아지"를 바로 안다. 어텐션은 "그것"이 다른 단어를 얼마나 참고할지를 숫자로 계산한다.

"그것"이 각 단어에 주는 어텐션 가중치 (예시)

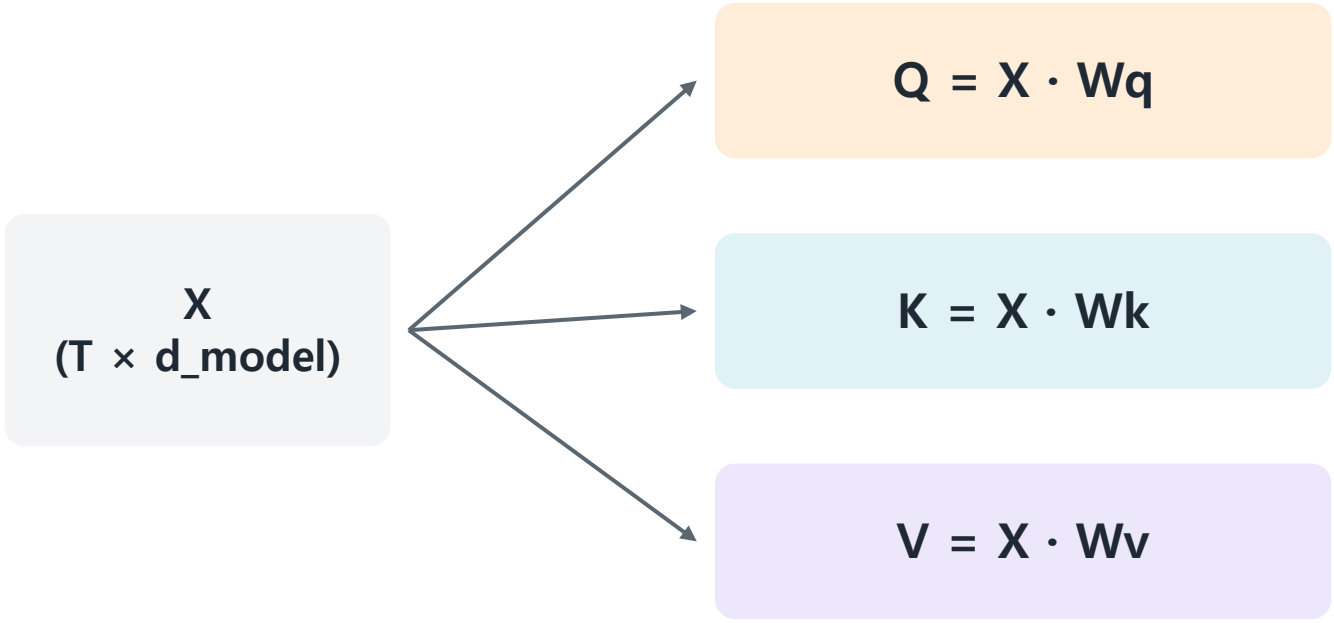


핵심

합계 = 1.00
(비율이 된다)

큰 가중치의 단어 정보를 많이 섞는다.

1단계: 입력 x에서 Q, K, V 만들기



알아 둘 점

- Wq, Wk, Wv는 학습으로 바뀌는 가중치(파라미터)
- 같은 입력 X에서 서로 다른 세 가지 "관점"을 만든다.
- shape: (T, d_model) × (d_model, d_k) = (T, d_k)
- 셀프(self) 어텐션: Q, K, V가 모두 같은 문장에서 나와서 붙은 이름

2단계: 점수 = Q와 K의 내적 (유사도)

내적(dot product)이란?

a = [1, 2, 3], b = [4, 5, 6]
a · b = 1×4 + 2×5 + 3×6 = 32

방향이 비슷하면 크고, 반대면 작다(음수).

모든 쌍을 한 번에: $Q \cdot K^T$

(T, d_k) × (d_k, T) = (T, T)

(i, j) 칸 = "i번 단어의 질문이
j번 단어의 이름표와 얼마나 맞나"

점수 행렬 (T = 3 예시)

	나는	AI를	배운다
나는	2.1	0.3	0.8
AI를	0.4	1.9	1.2
배운다	0.9	1.1	2.4

대각선(자기 자신)이 크게 나오는 경향이 있다. 행 하나 = 한 단어의 "참고 점수표".

왜 $\sqrt{d_k}$로 나눌까?

차원(d_k)이 클수록 내적 값의 크기도 커진다. 값이 너무 크면 softmax가 한 칸만 1에 가깝게 쏠려 학습이 멈춘다.

d_k	내적 값의 표준편차	$\sqrt{d_k}$ 로 나눈 뒤
4	약 2	약 1
64	약 8	약 1
512	약 22.6	약 1

(Q, K 성분이 평균 0, 분산 1일 때의 통계적 크기)

나누지 않으면

softmax([20, 1, 0]) ≈ [1, 0, 0]
→ 기울기가 거의 0
→ 학습이 진행되지 않는다.

그래서

$scores = Q \cdot K^T / \sqrt{d_k}$ (크기를 항상 일정하게 유지하는 안전장치)

3단계: softmax로 점수를 확률(비율)로

softmax 공식

$$\text{softmax}(z)_i = \exp(z_i) / \sum_j exp(z_j)$$

① 모두 양수로 ② 합이 1이 되게

예: 점수 [2.0, 1.0, 0.1]

점수	2.0	1.0	0.1
exp	7.39	2.72	1.11
확률	0.66	0.24	0.10

합 = 7.39 + 2.72 + 1.11 = 11.22 → 7.39 / 11.22 = 0.66

수치 안정화 요령

exp(1000)은 컴퓨터가 터진다(overflow).
최댓값을 먼저 빼도 결과는 같다:
softmax(z) = softmax(z − max(z))

행(row) 단위로 적용

점수 행렬 (T, T)의 각 행에 softmax를 적용
→ 한 단어가 모든 단어에 주는 비율이 된다.
(axis = −1)

4단계: 비율대로 V를 섞는다 (손으로 계산해 보기)

X = Q = K = V = [[1,0], [0,1], [1,1]] (토큰 3개, d_k = 2)

① 점수 $Q \cdot K^T / \sqrt{2}$

0.71	0	0.71
0	0.71	0.71
0.71	0.71	1.41

② softmax

0.40	0.20	0.40
0.20	0.40	0.40
0.25	0.25	0.50

③ 출력 = 가중치 · V

0.80	0.60
0.60	0.80
0.75	0.75

첫 번째 토큰 계산 (주황색 행)

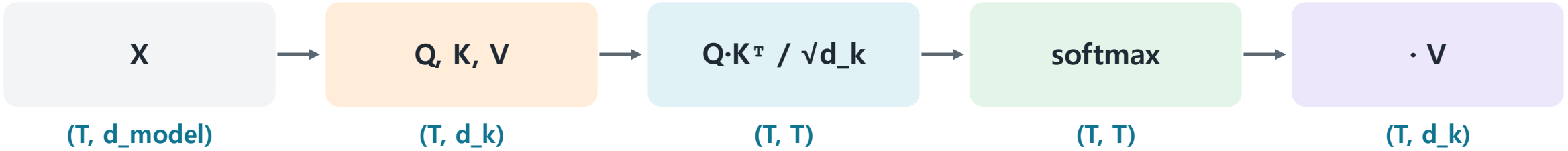
$출력_0 = 0.40 \times [1,0] + 0.20 \times [0,1] + 0.40 \times [1,1]$

$= [0.40, 0] + [0, 0.20] + [0.40, 0.40] = [0.80, 0.60]$

→ 첫 단어는 자기 자신과 세 번째 단어를 40%씩, 두 번째를 20% 섞은 새로운 벡터가 되었다.

어텐션 공식 총정리와 shape 추적

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^{T} / \sqrt{d_k} \right) \cdot V$$



입력과 출력의 shape이 같다 (T, ·) — 단어 수는 그대로, 각 벡터가 "문맥을 반영한 새 벡터"로 바뀐다.
디버깅 팁: shape을 매 단계 print 하면 대부분의 버그가 바로 보인다.

[코드 Step 3] 어텐션 함수 (NumPy)

numpy_version/step3_attention.py

```
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)  # overflow 방지
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)        # (T, T) 점수
    if mask is not None:
        scores = np.where(mask, -1e9, scores)  # 못 보는 칸
    weights = softmax(scores, axis=-1)     # 각 행의 합 = 1
    return weights @ V, weights            # 비율대로 V 섞기

# 사용: x에서 Q, K, V를 만든 뒤 호출
Q, K, V = x @ Wq, x @ Wk, x @ Wv
out, w = scaled_dot_product_attention(Q, K, V)
```

코드 읽는 법

① softmax: 최댓값을 빼 두어도 결과는 같고 overflow만 막힌다.

② Q @ K.T: 모든 단어쌍의 점수를 행렬곱 한 번으로 계산한다.

③ / np.sqrt(d_k): 점수의 크기를 일정하게 유지한다.

④ mask의 True 칸을 −1e9로 채우면 softmax 후 확률이 0이 된다.

⑤ weights @ V: 비율(가중치)대로 Value를 섞어 최종 출력을 만든다.

인과 마스크(Causal Mask): 미래는 볼 수 없다

글을 쓰는 모델(GPT)은 "다음 단어"를 맞혀야 한다. 정답이 될 미래 단어를 미리 보면 시험지를 훔쳐보는 것!

허용(1) / 차단(0)

1	0	0	0
1	1	0	0
1	1	1	0
1	1	1	1

i번째 단어는 자기 자신과 앞 단어(≤ i)만 참고

구현 방법

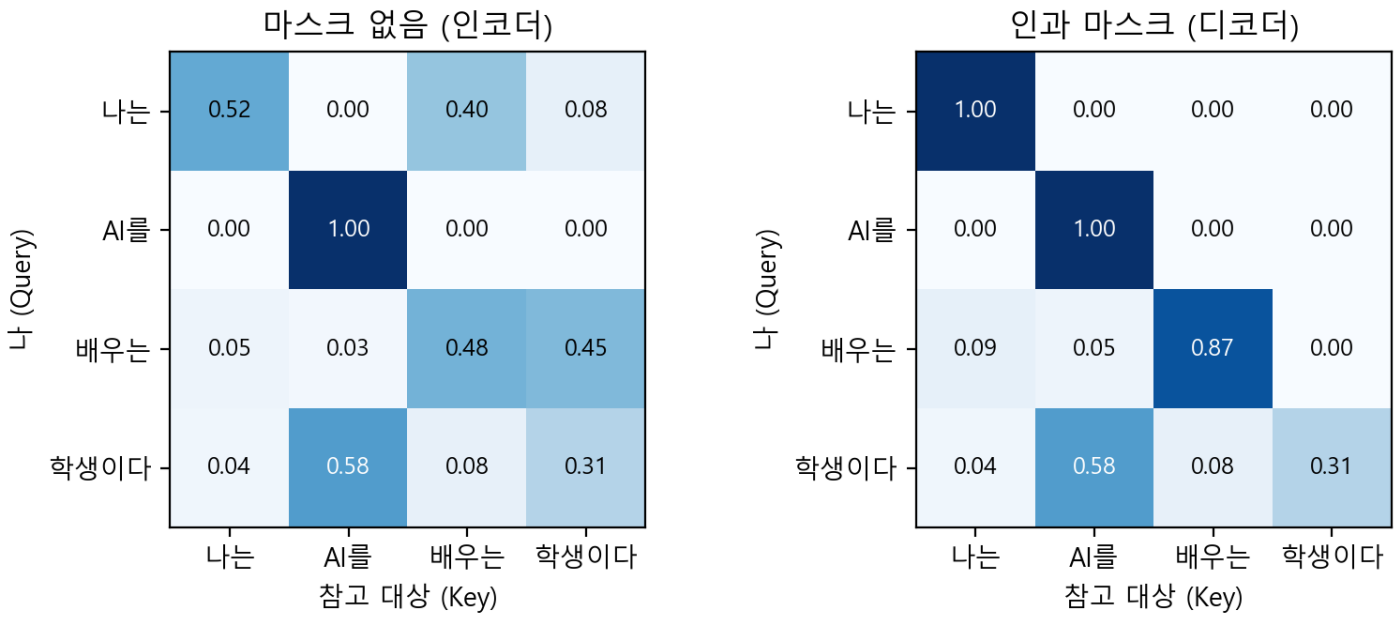
① 위쪽 삼각형(대각선 위)이 True인 마스크 생성
mask = np.triu(np.ones((T,T), bool), k=1)

② softmax 전에 True 칸을 −1e9로 덮어쓰기

③ softmax(−1e9) ≈ 0 → 가중치 0

※ 번역의 인코더처럼 문장 전체를 보는 경우엔 마스크를 쓰지 않는다.

실제 어텐션 가중치 보기: 마스크 전과 후



읽는 법

행 = 질문하는 단어(Q)
열 = 참고되는 단어(K)
진할수록 많이 참고

오른쪽: 위쪽 삼각형이 0이 되어 "과거만" 본다.

```
out, w = scaled_dot_product_attention(
    Q, K, V, mask=causal_mask(T))
assert np.allclose(np.triu(w, k=1), 0)
```

assert 로 "대각선 위쪽은 전부 0"임을 검산한다.

2교시 정리 퀴즈

Q1 위치 인코딩을 임베딩에 "더하는" 이유는?

Q2 Q, K, V는 각각 어떤 역할인가?

Q3 인과 마스크가 없으면 어떤 문제가 생길까?

→ 10분 휴식 후 3교시: 멀티헤드와 블록 조립

3교시

멀티헤드와 트랜스포머 블록

멀티헤드 어텐션 · FFN · 잔차연결 · LayerNorm · 블록 쌓기

멀티헤드 어텐션: 여러 관점으로 동시에 본다

한 번의 어텐션은 "한 가지 관계"만 보기 쉽다. 여러 개의 작은 어텐션(head)을 나란히 돌려 서로 다른 관계를 잡는다.

Head 1

문법 관계
주어 ↔ 동사

Head 2

가까운 단어
바로 앞·뒤 단어

Head 3

의미 관계
대명사 ↔ 대상

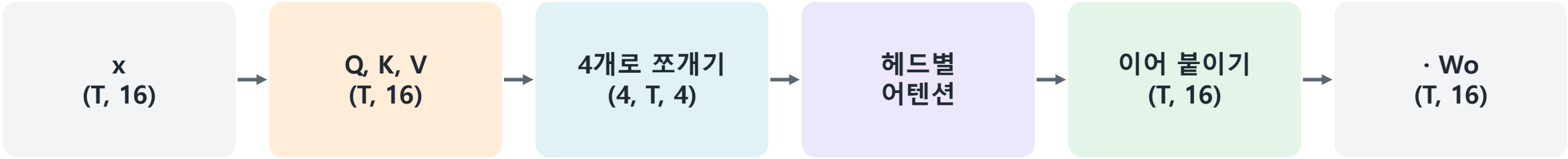
Head 4

기타 패턴
학습이 알아서 결정

비용은 그대로

d_model = 32, 헤드 4개 → 헤드마다 d_k = 32 / 4 = 8 차원만 담당.
벡터를 쪼개 쓰므로 계산량은 헤드 1개 때와 거의 같다. 각 헤드의 역할은 정해 주지 않아도 학습이 만든다.

멀티헤드 계산 과정: 쪼개고 → 각자 어텐션 → 이어 붙이기



예: d_model = 16, 헤드 4개 → d_k = 4

- 쪼개기: 열 방향으로 4조각 [0:4], [4:8], [8:12], [12:16]
- 각 헤드는 서로 독립적으로 Step 3의 어텐션을 수행
- 이어 붙이기(concat): 헤드 결과를 옆으로 붙여 다시 (T, 16)
- Wo: 헤드들의 결과를 섞어 주는 마지막 선형변환
- 입력과 출력 shape이 같아서 층을 쌓을 수 있다

[코드 Step 4] 멀티헤드 어텐션 (NumPy)

numpy_version/step4_multihead.py

```
def multi_head_attention(x, Wq, Wk, Wv, Wo, n_heads, causal=True):
    T, d_model = x.shape
    d_k = d_model // n_heads              # 헤드 하나의 길이
    Q, K, V = x @ Wq, x @ Wk, x @ Wv      # 각 (T, d_model)
    mask = causal_mask(T) if causal else None

    heads = []
    for h in range(n_heads):
        s = slice(h * d_k, (h + 1) * d_k)    # h번째 열 범위
        out_h, _ = scaled_dot_product_attention(
            Q[:, s], K[:, s], V[:, s], mask)
        heads.append(out_h)               # 각 (T, d_k)

    concat = np.concatenate(heads, axis=-1)  # (T, d_model)
    return concat @ Wo                    # 헤드 섞기
```

코드 읽는 법

① d_k = d_model // n_heads : 나누어떨어져야 한다.

② Wq, Wk, Wv는 모든 헤드 몫을 한 행렬에 모아 둔 것 (d_model × d_model).

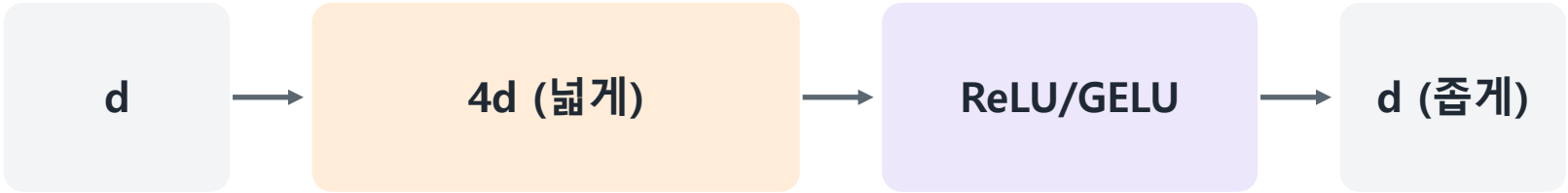
③ slice로 열을 잘라 헤드별로 어텐션 → Step 3 함수를 그대로 재사용.

④ np.concatenate: 헤드 결과를 옆으로 이어 붙인다.

⑤ 검산: n_heads=1 이면 Step 3 결과 @ Wo 와 정확히 같다.

피드포워드 네트워크(FFN): 각 단어가 혼자 생각하는 시간

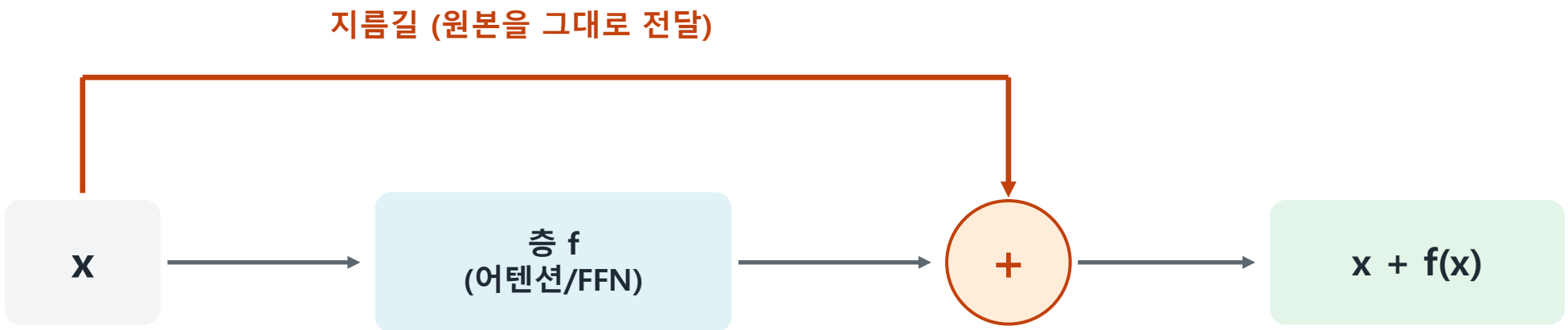
어텐션 = 단어끼리 "회의"하며 정보 교환 / FFN = 각자 돌아가서 "개인 정리"



- Linear(d → 4d) → 비선형 함수 → Linear(4d → d)
- 모든 위치(토큰)에 똑같은 가중치를 따로따로 적용
- 비선형 함수가 없으면 층을 아무리 쌓아도 하나의 선형 변환과 같다
- 트랜스포머 파라미터의 절반 이상이 FFN에 있다

```
def __call__(self, x):
    return self.l2(self.l1(x).relu())
# l1: d -> 4d,  l2: 4d -> d
```

잔차연결(Residual): 지름길을 만들어 준다



왜 필요할까?

층이 깊어지면 학습 신호(기울기)가 사라진다.
지름길 덕분에 기울기가 곧장 아래층까지 전달된다.

의미

층은 "원본을 통째로 바꾸는 것"이 아니라
"원본에 보탤 수정분(f)"만 학습하면 된다.

LayerNorm: 벡터의 크기를 가지런히

공식 (토큰 벡터 하나마다)

$y = \gamma \cdot (x - \text{평균}) / \sqrt{(\text{분산} + \varepsilon)} + \beta$

γ, β는 학습되는 값 (처음엔 1, 0)

왜 필요할까?

층을 지나며 값이 커지거나 작아지면
학습이 불안정해진다. 평균 0, 분산 1로
맞추면 안정적이고 빨리 학습된다.

손계산: x = [2, 4, 6, 8]

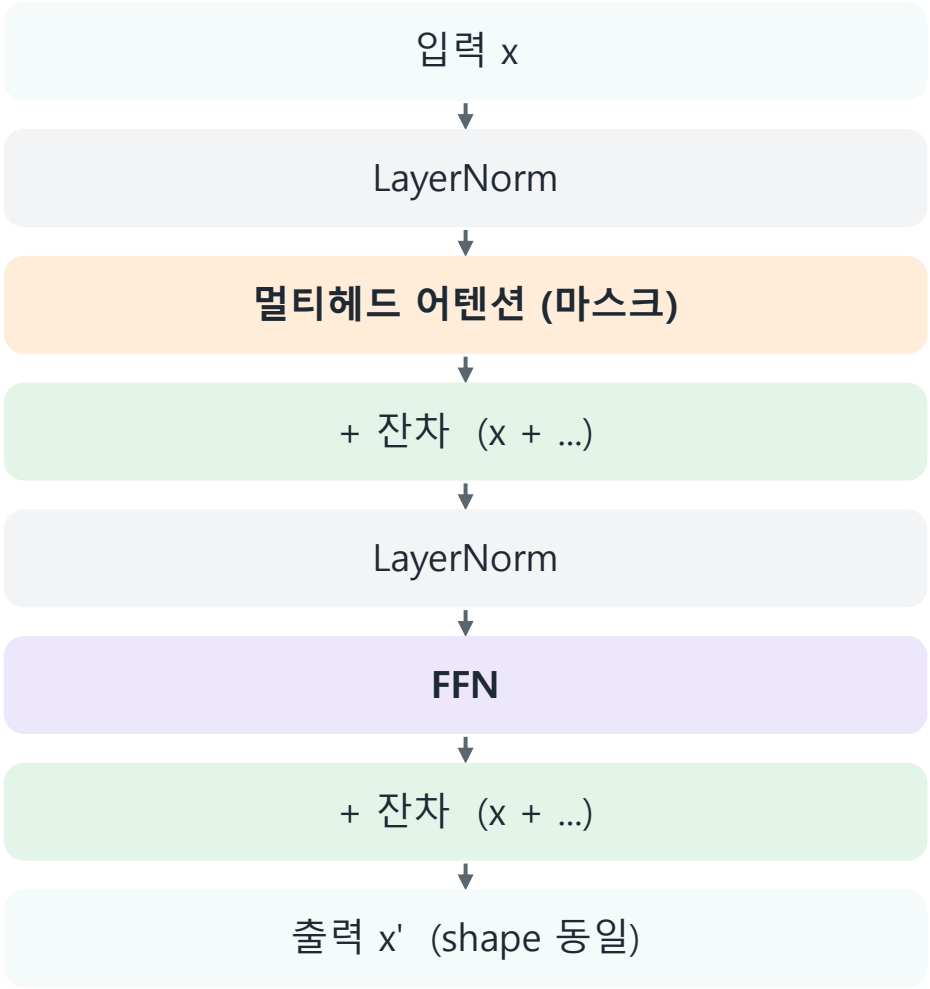
x	2	4	6	8
x − 5	−3	−1	1	3
÷ √5	−1.34	−0.45	0.45	1.34

평균 = 5, 분산 = (9+1+1+9)/4 = 5, 표준편차 = √5 ≈ 2.24

BatchNorm과 다른 점

LayerNorm은 "한 토큰 벡터 안"에서 계산
(마지막 축).
문장 길이·배치 크기에 영향받지 않아 언어모델에 적합하다.

트랜스포머 블록 조립



두 줄이면 끝

x = x + attn(ln1(x))

x = x + ffn(ln2(x))

역할 분담

어텐션: 단어 ↔ 단어 정보 교환

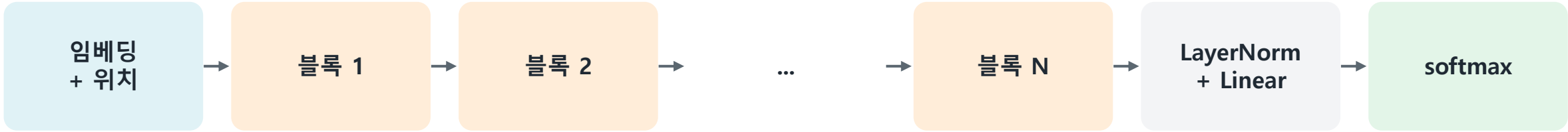
FFN: 단어 하나를 깊게 가공

잔차·정규화: 깊게 쌓아도 학습이 잘 되게

블록은 레고

입력·출력 shape이 같으므로 N개를 그대로 쌓을 수 있다. (GPT-3: 96개)

블록을 쌓고, 마지막에 "다음 토큰"을 예측



shape 흐름: (B, T) → (B, T, d) → ... → (B, T, d) → (B, T, V)

출력층의 의미

각 위치마다 어휘 V개의 점수(logits)를 낸다.
softmax → "다음 토큰이 각 글자일 확률".

예) "the " 다음 → 'm' 0.4, 's' 0.3, 'w' 0.1, ...

한 번에 T개를 학습

문장이 T글자라면 T개의 "다음 글자 맞히기" 문제가 한 번에 만들어진다. (마스크 덕분에 각 위치는 앞부분만 본다.)

1회차 마무리

오늘 배운 것

- 토큰화 · 임베딩 · 위치 인코딩
- Q, K, V 와 scaled dot-product attention
- 인과 마스크
- 멀티헤드, FFN, 잔차연결, LayerNorm
- 트랜스포머 블록의 구조

복습 과제 (30분)

numpy_version/step1~4 를 차례로 실행하고
shape 출력을 직접 확인해 보세요.

step3 의 가중치(Wq 등)를 바꾸면
어텐션 그림이 어떻게 달라지나요?

다음 시간 (2회차)

4교시 학습 원리와 자동미분 만들기 → 5교시 NumPy 트랜스포머 완성 → 6교시 PyTorch 버전과 실습 문제

REVIEW

2회차 시작: 지난 시간 복습

토큰·임베딩

글자 → 번호 → 벡터
(T, d_model)

위치 인코딩

sin/cos 표를 더해
순서 정보 추가

셀프 어텐션

$\text{softmax}(QK^T/\sqrt{d})\cdot V$
+ 인과 마스크

멀티헤드

쪼개서 따로 보고
이어 붙여 Wo로 섞기

블록

x+attn(ln(x))
x+ffn(ln(x))

오늘의 목표: 부품들을 "학습 가능한 모델"로 만들기

지금까지는 가중치가 랜덤이라 결과가 의미 없었다. 오늘은 ① 학습 원리 ② 기울기를 구하는 자동미분을 직접 만들고 ③ NumPy 트랜스포머를 완성해 실제로 글을 쓰게 한 뒤 ④ PyTorch로 같은 모델을 다시 만든다.

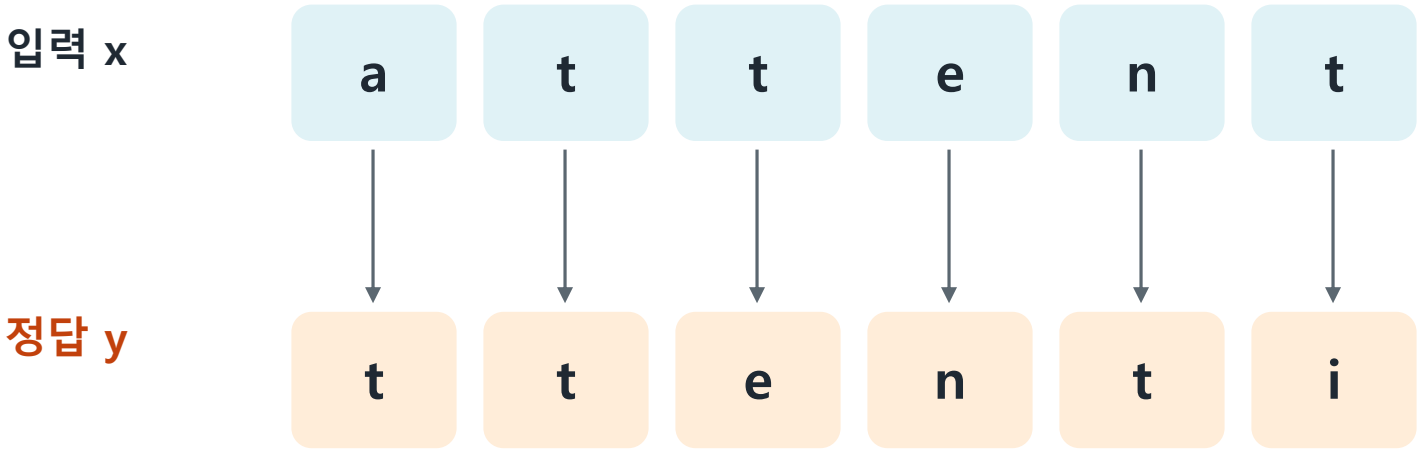
4교시

모델은 어떻게 배우나?

다음 글자 맞히기 · 손실 · 경사하강법 · 역전파 · 자동미분

학습 목표: "다음 글자 맞히기"

정답은 문장 안에 이미 있다! 입력을 한 칸 밀면 곧바로 정답지가 된다. (자기지도학습)



한 문장 = 여러 문제

"a" 다음은 "t", "at" 다음은 "t",
"att" 다음은 "e" … 6문제를 한 번에.

```
x = data[s : s+CTX]          # 입력 구간
y = data[s+1 : s+CTX+1]      # 한 칸 뒤 = 정답
```

손실 함수: 틀린 정도를 숫자 하나로

교차 엔트로피(Cross Entropy)

loss = – log(정답 글자에 준 확률)
여러 위치의 평균을 최종 loss로 쓴다.

정답에 준 확률	loss	의미
0.90	0.105	거의 맞힘
0.50	0.693	반반
0.10	2.303	많이 틀림
0.01	4.605	완전 틀림

학습 전 loss는 얼마일까?

아무것도 모르면 V개를 똑같이 찍는다.
loss = ln(V) = ln(28) ≈ 3.33
우리 실험의 첫 loss도 3.5 근처에서 출발!

목표

loss가 0에 가까워지도록
모든 가중치를 조금씩 조정한다.

그러려면 "어느 가중치를 어느 방향으로
얼마나" 바꿀지 알아야 한다 → 기울기

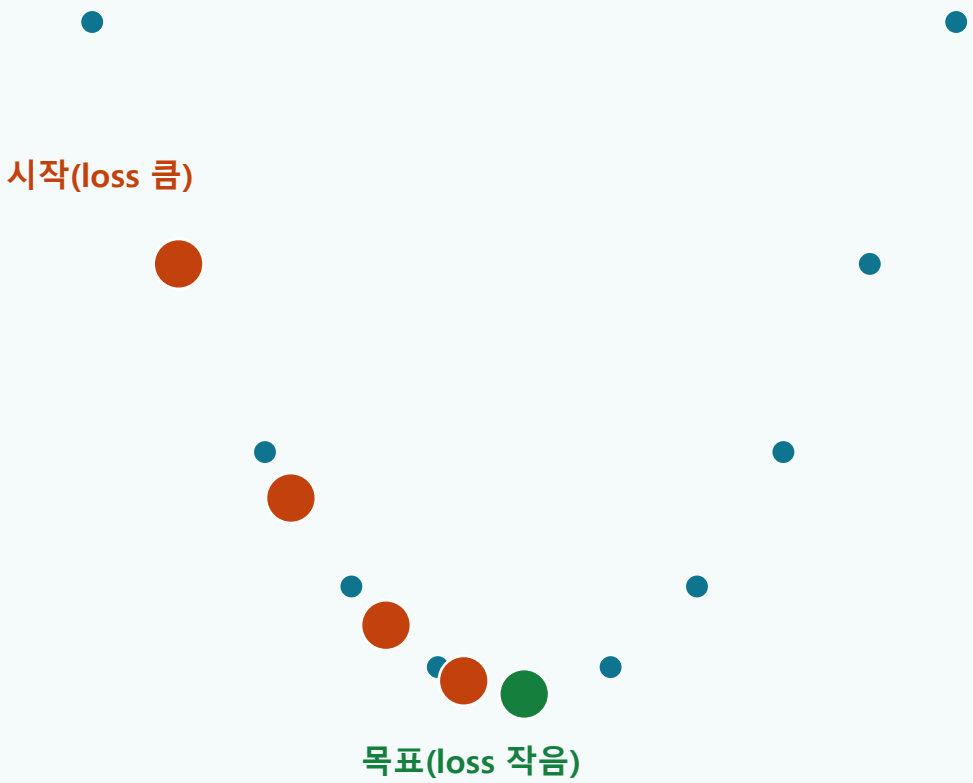
경사하강법: 안개 낀 산을 내려오기

한 줄 규칙

$$w \leftarrow w - lr \times (\partial \text{loss} / \partial w)$$

기울기$(\partial loss/\partial w)$의 반대 방향으로 조금 이동

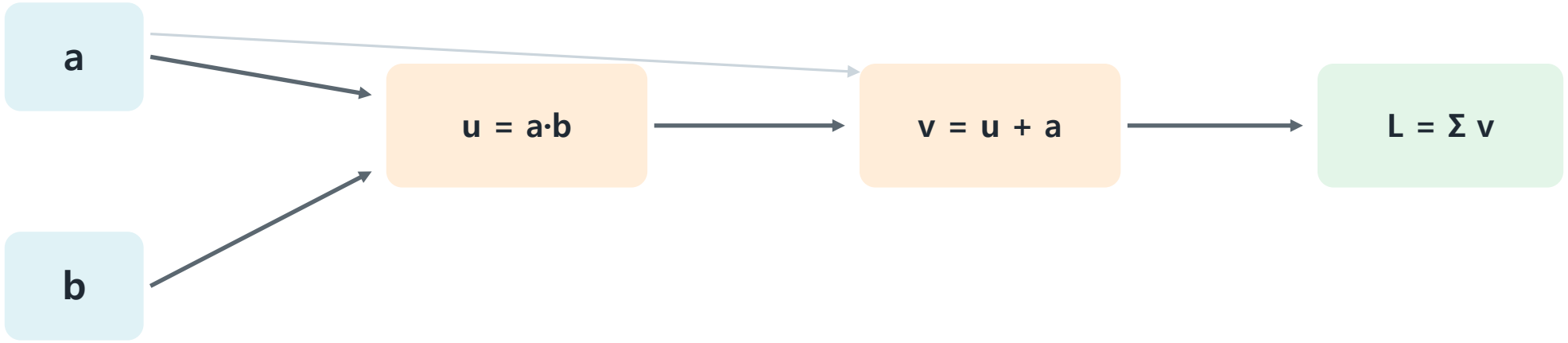
- 기울기: 내 위치에서 "가장 가파르게 올라가는 방향"
- 그 반대로 가면 loss가 줄어든다
- lr(학습률)이 너무 크면 튕겨 나가고, 너무 작으면 느리다
- Adam: 이동 방향·보폭을 과거 기울기의 평균으로 부드럽게 조절해 주는 똑똑한 버전



한 걸음마다 기울기 반대 방향으로

역전파와 연쇄법칙: 결과에서 거꾸로 기울기 전달

예: L = (a·b + a).sum() with a = [1,2,3], b = [4,5,6]



연쇄법칙(Chain Rule)

∂L/∂a = (∂L/∂v)·(∂v/∂u)·(∂u/∂a) + (∂L/∂v)·(∂v/∂a)
= 1·b + 1 = b + 1 = [5, 6, 7]

핵심 아이디어

각 연산은 "자기 국소 미분"만 알면 된다.
나머지는 뒤에서 오는 기울기에 곱해서 전달!

자동미분 엔진 설계: Tensor가 기억하는 3가지

① data / grad

data: 실제 값(NumPy 배열)
grad: ∂L/∂(이 값)
처음엔 0, backward 때 채워진다.

② parents

이 값을 만든 입력들.
계산 그래프를 거슬러 올라갈
"길" 역할을 한다.

③ _backward()

"내 기울기를 입력들에게
나눠 주는 방법"을 담은 함수.
연산마다 다르다.

backward()가 하는 일

① 그래프를 위상 정렬(입력 먼저) ② loss.grad = 1 로 시작 ③ 결과 → 입력 순서로 각 노드의 _backward() 실행.
※ 같은 값이 여러 곳에 쓰이면 기울기를 더한다(+=).

[코드 Step 5-1] Tensor 클래스: 덧셈과 곱셈

numpy_version/step5_autograd.py

```
class Tensor:
    def __init__(self, data, parents=()):
        self.data = np.asarray(data, dtype=np.float64)
        self.grad = np.zeros_like(self.data)
        self.parents = parents
        self._backward = lambda: None

    def __mul__(self, other):
        other = Tensor.wrap(other)
        out = Tensor(self.data * other.data, (self, other))
        def _backward():                    # d(a*b)/da = b
            self.grad += unbroadcast(out.grad * other.data,
                                     self.data.shape)
            other.grad += unbroadcast(out.grad * self.data,
                                      other.data.shape)
        out._backward = _backward
        return out
```

코드 읽는 법

① 계산 결과 out 은 자신을 만든 입력(self, other)을 기억한다.

② _backward 는 클로저: out.grad 를 받아 입력들의 grad 에 더해 준다.

③ 곱셈의 국소 미분: a의 기울기 = (나온 기울기) × b, b의 기울기 = × a.

④ += : 한 값이 여러 번 쓰였다면 기울기가 합쳐져야 하기 때문.

⑤ unbroadcast: 브로드캐스트로 커진 기울기를 원래 shape 으로 합쳐 줌.

[코드 Step 5-2] 행렬곱, softmax 그리고 backward()

numpy_version/step5_autograd.py

```
def __matmul__(self, other):           # C = A @ B
    out = Tensor(self.data @ other.data, (self, other))
    def _backward():
        self.grad += unbroadcast(out.grad @ swap(other.data), ...)
        other.grad += unbroadcast(swap(self.data) @ out.grad, ...)
    ...

def softmax(self):
    ...                                # s = 확률
    def _backward():                   # dx = s*(dy - sum(dy*s))
        g = out.grad
        self.grad += s * (g - (g * s).sum(-1, keepdims=True))

def backward(self):
    order = [...]                      # 위상 정렬 (입력이 먼저)
    self.grad = np.ones_like(self.data)  # dL/dL = 1
    for t in reversed(order):          # 결과 -> 입력
        t._backward()
```

코드 읽는 법

① 행렬곱: $dA = dC \cdot B^T$, $dB = A^T \cdot dC$ (swap = 마지막 두 축 교환).

② softmax 역전파는 위 공식 한 줄이면 충분하다.

③ 그 외 add, mul, pow, sum, reshape, transpose, relu, masked_fill, take(임베딩), cross_entropy 를 같은 방식으로 구현.

④ backward(): 위상 정렬 후 reversed 순서로 _backward 호출.

⑤ 전체 파일은 약 200줄. PyTorch autograd 의 축소판이다!

기울기 검사(Gradient Check): 내가 만든 미분이 맞나?

수치 미분

f'(w) ≈ [f(w+h) − f(w−h)] / 2h

h를 아주 작게(1e-6) 잡고 값을 살짝 흔들어서
실제로 loss가 얼마나 변하는지 측정

실행 결과

최대 오차 ≈ 1.4e-10
기울기 검사 통과!

오차가 1e-6 보다 작으면 올바르게 구현된 것

```
loss = loss_fn(); loss.backward()     # 내가 만든 기울기
num = np.zeros_like(W.data)
for i in np.ndindex(*W.data.shape):
    old = W.data[i]
    W.data[i] = old + h; lp = loss_fn().data
    W.data[i] = old - h; lm = loss_fn().data
    W.data[i] = old
    num[i] = (lp - lm) / (2 * h)      # 수치 미분
assert np.allclose(num, W.grad, atol=1e-6)
```

실무 팁

새 연산을 만들 때마다
기울기 검사를 하는 습관!
(느려서 학습엔 쓰지 않는다)

5교시

NumPy 트랜스포머 완성

Step 6: 모든 부품을 조립해 학습시키고 글을 쓰게 한다

조립 설계도: 클래스 하나 = 부품 하나

Linear	y = xW + b
LayerNorm	정규화 (γ, β)
MultiHeadAttention	q,k,v,o 4개의 Linear
FeedForward	Linear 2개 + ReLU
Block	ln1·attn·ln2·ffn
GPT	emb + Block×N + head
Adam	가중치 갱신

아래층(Linear)부터 위층(GPT)으로 한 단계씩 쌓아 올린다.

공통 약속

- __call__(x): 순전파
- params(): 학습할 Tensor 목록

모든 부품이 같은 규칙을 가지므로 조립이 쉽고, Adam은 params() 목록만 받으면 된다.

[코드 Step 6-1] Linear 와 LayerNorm

numpy_version/step6_transformer.py

```
class Linear:
    def __init__(self, n_in, n_out, bias=True):
        self.W = Tensor(rng.normal(0, 1/np.sqrt(n_in),
                                   size=(n_in, n_out)))
        self.b = Tensor(np.zeros(n_out)) if bias else None
    def __call__(self, x):
        y = x @ self.W
        return y + self.b if self.b is not None else y

class LayerNorm:
    def __init__(self, d, eps=1e-5):
        self.gamma, self.beta = Tensor(np.ones(d)), Tensor(np.zeros(d))
        self.eps = eps
    def __call__(self, x):
        mu = x.mean(axis=-1, keepdims=True)
        xc = x - mu
        var = (xc * xc).mean(axis=-1, keepdims=True)
        return xc * ((var + self.eps) ** -0.5) * self.gamma + self.beta
```

코드 읽는 법

① 초기값 1/√n_in : 입력이 많을수록 작게 → 출력이 폭주하지 않는다.

② x @ W : Tensor의 @ 가 Step 5의 matmul(역전파 포함)을 부른다.

③ LayerNorm 은 새 연산을 만들지 않고 mean, −, ×, ** 만 조합했다.

④ 조합만 해도 자동미분이 알아서 기울기를 구해 준다!

[코드 Step 6-2] 멀티헤드 어텐션 (배치 + 자동미분)

numpy_version/step6_transformer.py

```
class MultiHeadAttention:
    def __init__(self, d_model, n_heads):
        self.h, self.dk = n_heads, d_model // n_heads
        self.q, self.k, self.v, self.o = (Linear(d_model, d_model) for _ in range(4))

    def __call__(self, x):                          # x: (B, T, D)
        B, T, D = x.data.shape
        def split(t):                               # (B,T,D)->(B,h,T,dk)
            return t.reshape(B, T, self.h, self.dk).transpose(0, 2, 1, 3)
        q, k, v = split(self.q(x)), split(self.k(x)), split(self.v(x))
        scores = (q @ k.transpose(0,1,3,2)) * (1.0/np.sqrt(self.dk))
        mask = np.triu(np.ones((T, T), dtype=bool), k=1)
        w = scores.masked_fill(mask, -1e9).softmax()
        ctx = (w @ v).transpose(0, 2, 1, 3).reshape(B, T, D)
        return self.o(ctx)
```

코드 읽는 법

① Step 4의 for 문 대신 헤드를 "새 축"으로 만들어 한 번에 계산한다.

② reshape → transpose 로 (B, h, T, dk) 모양을 만든다.

③ q @ k^T 는 (B, h, T, T): 모든 문장·헤드의 점수를 한 번에.

④ masked_fill → softmax 순서가 Step 3과 같다.

⑤ 다시 transpose → reshape 으로 헤드를 이어 붙이고 o 로 섞는다.

[코드 Step 6-3] FeedForward 와 Block

numpy_version/step6_transformer.py

```
class FeedForward:
    def __init__(self, d_model):
        self.l1 = Linear(d_model, 4 * d_model)
        self.l2 = Linear(4 * d_model, d_model)
    def __call__(self, x):
        return self.l2(self.l1(x).relu())

class Block:
    def __init__(self, d_model, n_heads):
        self.ln1 = LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ln2 = LayerNorm(d_model)
        self.ffn = FeedForward(d_model)
    def __call__(self, x):
        x = x + self.attn(self.ln1(x))   # 잔차연결
        x = x + self.ffn(self.ln2(x))
        return x
```

코드 읽는 법

① FFN : 4배로 넓혔다가 ReLU 후 원래 크기로.

② Block : 슬라이드에서 본 두 줄이 그대로 코드가 되었다.

③ x + … 의 덧셈이 잔차연결. 기울기가 지름길로도 흐른다.

④ 입력·출력 shape이 같아서 리스트에 담아 반복 호출할 수 있다.

[코드 Step 6-4] GPT 모델 전체

numpy_version/step6_transformer.py

```
class GPT:
    def __init__(self, vocab_size, d_model=32, n_heads=4,
                 n_layers=2, max_len=64):
        self.emb = Tensor(rng.normal(0, 0.1, (vocab_size, d_model)))
        self.pe = positional_encoding(max_len, d_model)  # 상수
        self.blocks = [Block(d_model, n_heads) for _ in range(n_layers)]
        self.ln_f = LayerNorm(d_model)
        self.head = Linear(d_model, vocab_size)
        self.scale = np.sqrt(d_model)

    def __call__(self, idx):                    # idx: (B, T) 정수
        B, T = idx.shape
        x = self.emb.take(idx) * self.scale + self.pe[:T]
        for blk in self.blocks:
            x = blk(x)
        return self.head(self.ln_f(x))          # (B, T, V) 점수
```

코드 읽는 법

① emb.take(idx): 번호로 임베딩 표에서 벡터 조회 (기울기는 해당 행에만).

② √d_model 을 곱해 위치 인코딩에 임베딩이 묻히지 않게 한다.

③ + self.pe[:T] : Step 2 의 위치 인코딩을 더한다 (학습 X).

④ 블록을 N번 통과 → ln_f → head 로 다음 글자 점수를 낸다.

[코드 Step 6-5] Adam 옵티마이저와 학습 루프

numpy_version/step6_transformer.py

```
class Adam:
    def step(self):
        self.t += 1
        for i, p in enumerate(self.p):
            self.m[i] = b1*self.m[i] + (1-b1)*p.grad        # 기울기 평균
            self.v[i] = b2*self.v[i] + (1-b2)*p.grad**2     # 크기 평균
            mh = self.m[i] / (1 - b1**self.t)               # 초반 보정
            vh = self.v[i] / (1 - b2**self.t)
            p.data -= self.lr * mh / (np.sqrt(vh) + eps)

for step in range(1, STEPS + 1):
    x, y = (랜덤 위치에서 CTX 글자 / 한 칸 뒤 글자)
    logits = model(x)                                       # 1 순전파
    loss = logits.reshape(-1, V).cross_entropy(y.reshape(-1))   # 2 손실
    opt.zero_grad()                                         # 3 기울기 비우기
    loss.backward()                                         # 4 역전파
    opt.step()                                              # 5 갱신
```

코드 읽는 법

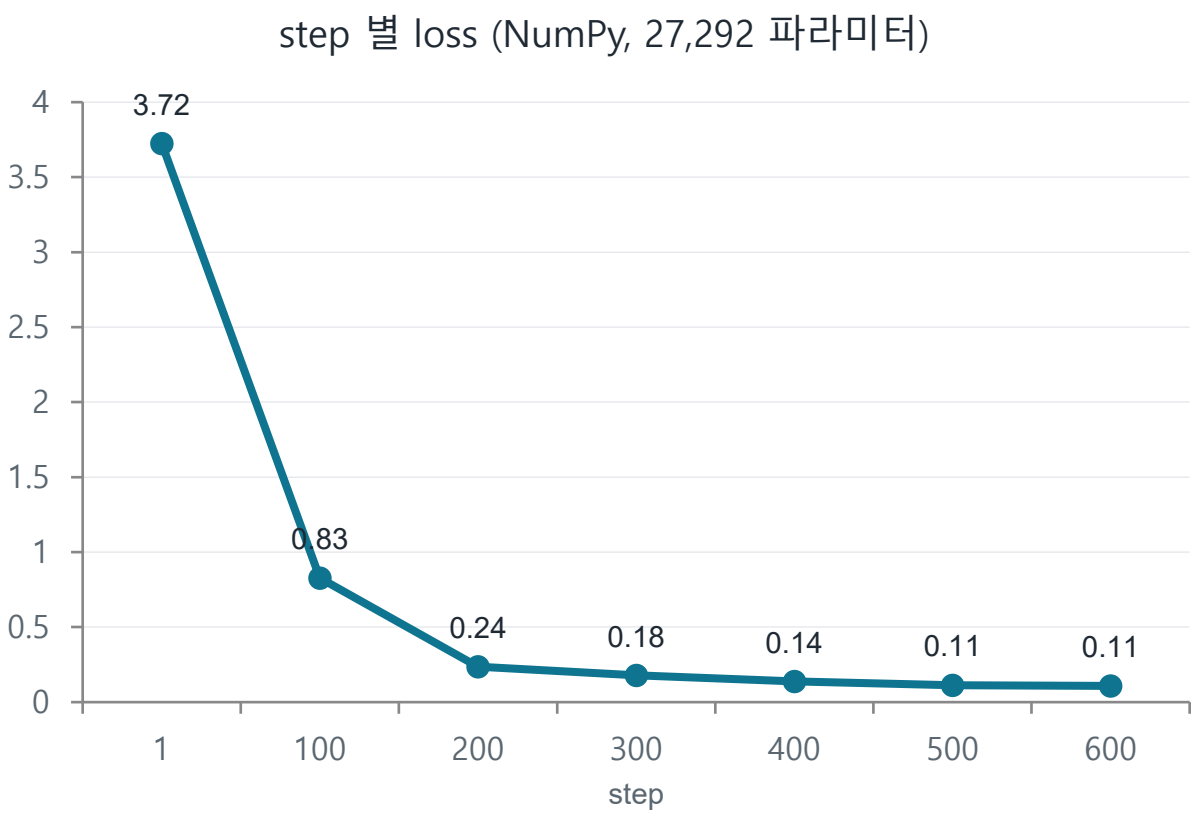
① Adam: 과거 기울기의 평균(m)과 크기(v)로 파라미터마다 보폭을 조절.

② 학습 루프는 5단계 리듬: 순전파 → 손실 → 비우기 → 역전파 → 갱신.

③ zero_grad 를 빼먹으면 기울기가 계속 누적된다(자주 하는 실수!).

④ 이 5단계는 PyTorch 에서도 똑같다.

학습 결과: loss가 내려가고 글을 쓴다



3.72 → 0.11 : 랜덤이던 가중치가 "규칙"을 학습한 증거

생성 결과 (일부)

the sentence in many ways.
layers stack up and the meaning
grows richer.
little by little the model learns
to predict the ne...

해석

학습 글(약 300자)을 거의 외웠다. 의미 있는 글을 쓰려면 더 많은 데이터가 필요하다.

모델 크기와 파라미터 수 세어 보기

부품 (d=32, V=28)	계산식	개수
임베딩	V × d = 28 × 32	896
어텐션 (q,k,v,o)	4 × (32×32 + 32)	4,224
FFN	(32×128+128) + (128×32+32)	8,352
LayerNorm ×2	2 × (32+32)	128
블록 1개 합계		12,704
블록 2개 + ln_f + head	2×12,704 + 64 + 924	26,396
전체	896 + 26,396	27,292

코드와 일치!

step6 실행 시 출력되는
"파라미터 수: 27292"와 같다.

규모 비교

우리 모델: 약 2.7만 개
GPT-2 small: 약 1.2억 개
GPT-3: 약 1,750억 개

구조는 같고 크기만 다르다.

6교시

PyTorch로 실전 버전

같은 모델을 현실적인 방식으로: nn.Module · 자동미분 · GPU

NumPy 버전 vs PyTorch 버전

항목	NumPy (직접 구현)	PyTorch (실전)
미분	Tensor 클래스 + _backward 직접 작성	loss.backward() 한 줄 (autograd)
부품	Linear, LayerNorm 직접 구현	nn.Linear, nn.LayerNorm, nn.Embedding
옵티마이저	Adam 직접 구현	torch.optim.AdamW
속도	CPU, 작은 모델만 가능	GPU 가속, 큰 모델 가능
장점	동작 원리를 100% 이해	짧고 빠르고 검증됨, 업계 표준

원리를 이해한 뒤에 도구를 쓰면, PyTorch의 모든 줄이 "아, 이거 우리가 직접 만든 거!"로 읽힌다.

우리가 만든 것 ↔ PyTorch 부품 대응표

NumPy 버전	PyTorch 버전
`E[ids]` (임베딩 조회)	`nn.Embedding(V, d)`
`positional_encoding()`	`register_buffer("pe", pe)`
`x @ W + b`	`nn.Linear(d_in, d_out)`
`LayerNorm` 클래스	`nn.LayerNorm(d)`
`masked_fill(mask, -1e9)`	`masked_fill(mask, float('-inf'))`
`Tensor.backward()`	`loss.backward()`
`Adam` 클래스	`torch.optim.AdamW(model.parameters())`
`params()` 목록	`model.parameters()` (자동 수집)

[PyTorch Step 1] 임베딩 + 위치 인코딩

pytorch_version/step1_embedding_pe.py

```
class TokenAndPosition(nn.Module):
    def __init__(self, vocab_size, d_model, max_len):
        super().__init__()
        self.d_model = d_model
        self.tok = nn.Embedding(vocab_size, d_model)
        pos = torch.arange(max_len).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2)
                        * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe)   # 학습 안 하는 상수

    def forward(self, idx):                    # idx: (B, T)
        T = idx.shape[1]
        x = self.tok(idx) * math.sqrt(self.d_model)
        return x + self.pe[:T]                 # (B, T, d_model)
```

코드 읽는 법

① nn.Module 을 상속하면 안에 만든 파라미터를 PyTorch 가 자동으로 모아 준다.

② nn.Embedding 이 NumPy 의 E[ids] 조회를 대신한다.

③ register_buffer: 파라미터는 아니지만 모델과 함께 GPU 이동·저장되는 상수.

④ (T, d) 를 (B, T, d) 에 더하면 B 방향으로 자동 브로드캐스트.

[PyTorch Step 2~3] 멀티헤드 어텐션

pytorch_version/step3_multihead.py

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, max_len, dropout=0.0):
        super().__init__()
        self.h, self.dk = n_heads, d_model // n_heads
        self.q = nn.Linear(d_model, d_model, bias=False)
        self.k = nn.Linear(d_model, d_model, bias=False)
        self.v = nn.Linear(d_model, d_model, bias=False)
        self.o = nn.Linear(d_model, d_model)
        self.drop = nn.Dropout(dropout)
        self.register_buffer("causal", torch.tril(torch.ones(max_len, max_len)))

    def forward(self, x):                              # (B, T, D)
        B, T, D = x.shape
        split = lambda t: t.view(B, T, self.h, self.dk).transpose(1, 2)
        q, k, v = split(self.q(x)), split(self.k(x)), split(self.v(x))
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.dk)
        scores = scores.masked_fill(self.causal[:T, :T] == 0, float("-inf"))
        w = self.drop(F.softmax(scores, dim=-1))
        ctx = (w @ v).transpose(1, 2).contiguous().view(B, T, D)
        return self.o(ctx)
```

코드 읽는 법

① NumPy 버전과 줄 단위로 같은 흐름이다. 달라진 건 도구뿐!

② tril: 아래 삼각형이 1인 행렬 → 0인 칸(미래)을 −inf 로 가린다.

③ view + transpose 로 (B, h, T, dk) 를 만들어 모든 헤드를 병렬 계산.

④ contiguous(): transpose 후 메모리를 정리해야 view 를 쓸 수 있다.

⑤ dropout: 학습 중 일부 연결을 꺼서 과적합을 막는다.

[PyTorch Step 4] 트랜스포머 블록

pytorch_version/step4_block.py

```
class FeedForward(nn.Module):
    def __init__(self, d_model, dropout=0.0):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),                          # 부드러운 ReLU
            nn.Linear(4 * d_model, d_model),
            nn.Dropout(dropout))
    def forward(self, x): return self.net(x)

class Block(nn.Module):
    def __init__(self, d_model, n_heads, max_len, dropout=0.0):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads, max_len, dropout)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = FeedForward(d_model, dropout)
    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x
```

코드 읽는 법

① nn.Sequential: 층을 순서대로 이어 붙이는 상자.

② GELU: ReLU 와 비슷하지만 곡선이 부드러워 GPT 계열이 즐겨 쓴다.

③ Block.forward 의 두 줄은 NumPy 버전과 완전히 동일하다.

④ 실행 결과: 블록 하나의 파라미터 12,608개 (d=32, bias 차이로 NumPy 와 약간 다름).

[PyTorch Step 5-1] GPT 모델과 글 생성

pytorch_version/step5_gpt_train.py

```
class GPT(nn.Module):
    def __init__(self, vocab_size, d_model=64, n_heads=4,
                 n_layers=3, max_len=64, dropout=0.1):
        super().__init__()
        self.embed = TokenAndPosition(vocab_size, d_model, max_len)
        self.blocks = nn.ModuleList(
            Block(d_model, n_heads, max_len, dropout) for _ in range(n_layers))
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size)
    def forward(self, idx, targets=None):
        x = self.embed(idx)
        for blk in self.blocks: x = blk(x)
        logits = self.head(self.ln_f(x))                # (B, T, V)
        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)),
                                   targets.view(-1))
        return logits, loss
```

코드 읽는 법

① nn.ModuleList: 블록 목록을 모델의 일부로 등록(파라미터 자동 수집).

② F.cross_entropy: softmax + −log 을 한 번에, 수치적으로도 안전.

③ view(-1, V): (B, T, V) 를 (B·T, V) 로 펴서 모든 위치의 손실을 평균.

④ targets 를 주면 학습용(loss 반환), 안 주면 추론용으로 쓸 수 있다.

[PyTorch Step 5-2] 학습 루프

pytorch_version/step5_gpt_train.py

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = GPT(len(chars), d_model=64, n_heads=4, n_layers=3,
            max_len=CTX).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=0.01)

model.train()                                  # dropout 켜기
for step in range(1, STEPS + 1):
    x, y = get_batch()
    _, loss = model(x, y)                      # 1 순전파 + 손실
    opt.zero_grad(set_to_none=True)            # 2 기울기 비우기
    loss.backward()                            # 3 역전파 (자동미분)
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # 4 클리핑
    opt.step()                                 # 5 갱신

model.eval()                                   # dropout 끄기
```

코드 읽는 법

① 리듬은 NumPy 버전과 같다. 달라진 점은 backward 한 줄!

② .to(device): 모델과 데이터를 GPU 로 보내 수십 배 빠르게.

③ clip_grad_norm_: 기울기가 너무 크면 잘라 학습 폭주를 막는다.

④ train()/eval(): dropout 같은 층의 동작을 학습용/추론용으로 전환.

글 생성: temperature 와 top-k

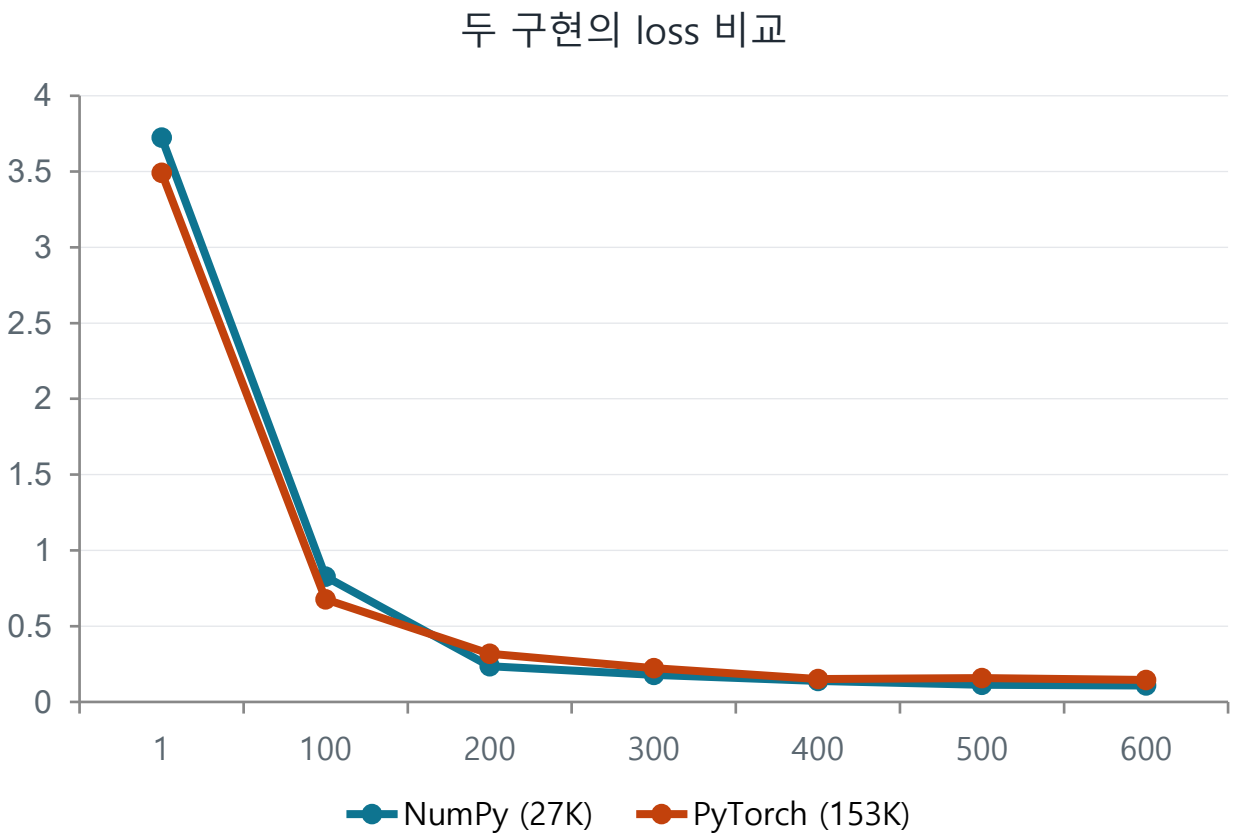
pytorch_version/step5_gpt_train.py

```
@torch.no_grad()                          # 기울기 계산 불필요 -> 빠르고 메모리 절약
def generate(self, idx, n_new, temperature=0.8, top_k=None):
    for _ in range(n_new):
        logits, _ = self(idx[:, -self.max_len:])  # 최근 토큰만
        logits = logits[:, -1, :] / temperature   # 마지막 위치 점수
        if top_k is not None:
            kth = torch.topk(logits, top_k).values[:, -1, None]
            logits = logits.masked_fill(logits < kth, float('-inf'))
        probs = F.softmax(logits, dim=-1)
        nxt = torch.multinomial(probs, 1)         # 확률대로 뽑기
        idx = torch.cat([idx, nxt], dim=1)        # 뒤에 이어 붙이기
    return idx
```

설정	효과
temperature 낮음 (0.3)	가장 그럴듯한 글자만, 안정적·반복적
temperature 높음 (1.5)	다양하지만 엉뚱한 글자도 나옴
top-k = 5	상위 5개 후보 안에서만 뽑는다

생성(자기회귀): 한 글자 예측 → 입력 뒤에 붙임 → 다시 예측. ChatGPT가 글을 쓰는 방식과 같다. (No-grad: 학습이 아니라 사용할 때)

PyTorch 학습 결과와 검증



검증: 내 구현 = 공식 구현

step6_compare_builtin.py
가중치를 nn.MultiheadAttention에
복사해 넣으면 출력 차이 약 6e-8
→ 올바르게 구현했다는 증거

관찰

두 구현 모두 loss가 비슷하게 감소.
(모델 크기·dropout이 달라 값은 조금씩 다르다)

실제 서비스에선 nn.TransformerEncoderLayer 같은 검증된 내장 부품을 쓰기도 한다.

진짜 ChatGPT와의 차이: 어디를 키우면 될까?

데이터

우리: 300글자
실제: 수조 토큰의 웹·책·코드

토크나이저

우리: 문자 단위
실제: BPE 서브워드(수만 어휘)

모델 크기

우리: 2.7만~15만 파라미터
실제: 수십억~수천억

학습 후 정렬

사전학습 후 SFT·RLHF 등으로
대화·안전하게 다듬는다

추론 효율

KV 캐시, 양자화, 병렬화로
빠르고 싸게 서비스

긴 문맥

RoPE 같은 위치 방식,
효율적 어텐션으로 확장

핵심 구조(어텐션 + 블록)는 오늘 만든 것과 같다. 나머지는 규모와 엔지니어링의 문제!

SUMMARY

전체 요약: 트랜스포머를 한 장으로

1	**토큰화·임베딩**	글자 → 번호 → 벡터 (T, d)
2	**위치 인코딩**	sin/cos 를 더해 순서 정보 추가
3	**셀프 어텐션**	softmax(QK^T/√d)·V, 마스크로 미래 차단
4	**멀티헤드**	여러 관점을 병렬로 보고 합친다
5	**블록**	x + attn(ln(x)), x + ffn(ln(x))
6	**출력·학습**	다음 토큰 확률 → 교차 엔트로피 → 역전파 → Adam

NumPy로 원리를, PyTorch로 실전을. 두 버전 모두 직접 돌려 본 당신은 이제 트랜스포머를 "만들 수 있는" 사람입니다!

PRACTICE

실습 문제

스스로 풀어 본 뒤, 다음 슬라이드의 솔루션과 비교하세요

PRACTICE

실습 문제 1 ~ 2

문제 1. 손계산 어텐션

토큰 3개, d_k = 2, 마스크 없음.
Q = K = V = [[1,0], [0,1], [1,1]]

(1) 첫 번째 토큰의 점수 $Q_0 \cdot K^T/\sqrt{2}$ 를 구하라.
(2) softmax 를 적용해 가중치를 구하라.
(3) 첫 번째 토큰의 출력 벡터를 구하라.
(4) 코드로 확인하라.

힌트: exp(0.707) ≈ 2.03, exp(0) = 1

문제 2. 파라미터 수 세기

NumPy 모델: d_model = 32, 헤드 4개,
블록 2개, 어휘 V = 28.

(1) 블록 1개의 파라미터 수는?
(2) 모델 전체의 파라미터 수는?
(3) step6 을 실행해 출력과 비교하라.

힌트: Linear(a, b) = a×b + b
임베딩과 출력층(head)도 잊지 말 것!

PRACTICE

실습 문제 3 ~ 4

문제 3. top-k 샘플링 (NumPy)

함수 top_k_sample(logits, k, rng) 을 구현하라.

- 점수가 높은 k개 글자만 후보로 남긴다.
- 나머지는 확률 0 으로 만든다.
- 남은 후보 중에서 확률대로 하나를 뽑는다.

검증: logits = [1, 3, 0.5, 2.5, −1], k = 2 로
1000번 뽑으면 1번과 3번만 나와야 한다.

힌트: np.sort, np.where, softmax(Step 3)

문제 4. 층 수 실험 (PyTorch)

pytorch_version 의 GPT 에서 n_layers 를
1, 2, 4 로 바꿔 300 step 학습하라.
(d_model=64, 헤드 4, dropout=0)

(1) 마지막 50 step 의 평균 loss 를 표로 정리
(2) 층이 많을수록 어떻게 되는가?
(3) 그 이유를 한 문장으로 설명하라.

힌트: 층이 많으면 표현력이 커진다.

SOLUTION

솔루션 1. 손계산 어텐션

풀이

(1) 점수 = $[Q_0 \cdot K_0, Q_0 \cdot K_1, Q_0 \cdot K_2] / \sqrt{2}$
$Q_0 = [1,0] \rightarrow [1, 0, 1] / 1.414$
$= [0.707, 0, 0.707]$

(2) exp = [2.03, 1.00, 2.03], 합 = 5.06
가중치 = [0.401, 0.198, 0.401]

(3) 출력 = 0.401·[1,0] + 0.198·[0,1] + 0.401·[1,1]
= [0.802, 0.599]

practice_solutions.py

```
X = np.array([[1.,0.],[0.,1.],[1.,1.]])
out, w = scaled_dot_product_attention(X, X, X)
print(w[0].round(3))     # [0.401 0.198 0.401]
print(out[0].round(3))   # [0.802 0.599]
```

확인 포인트

- 가중치의 합 = 1
- 자기 자신과 닮은 토큰(0번, 2번)에 큰 가중치가 갔다.

SOLUTION

솔루션 2. 파라미터 수

practice_solutions.py

```
d, V, L = 32, 28, 2
attn  = 4 * (d*d + d)                      # q,k,v,o      = 4,224
ffn   = (d*4*d + 4*d) + (4*d*d + d)        # 128->, ->32  = 8,352
ln    = 2 * (2*d)                           # gamma,beta x2 = 128
block = attn + ffn + ln                     # = 12,704
total = V*d + L*block + 2*d + (d*V + V)
#     임베딩  블록들   ln_f   출력층(head)
print(block, total)                         # 12704 27292
```

정답

(1) 블록 1개: 12,704
(2) 전체: 27,292
(3) step6 출력
"파라미터 수: 27292" 와 일치

자주 틀리는 부분

임베딩(896)과 출력층(924), 마지막 LayerNorm(64)을 빼먹기 쉽다. 어텐션의 q,k,v,o 모두 bias를 가진다(NumPy 버전).

SOLUTION

솔루션 3. top-k 샘플링

practice_solutions.py

```
def top_k_sample(logits, k, rng, temperature=1.0):
    logits = logits / temperature
    cut = np.sort(logits)[-k]                  # 상위 k번째 점수
    logits = np.where(logits < cut, -np.inf, logits)  # 탈락
    p = softmax(logits)                        # 탈락한 칸은 확률 0
    return int(rng.choice(len(p), p=p))

rng = np.random.default_rng(0)
logits = np.array([1.0, 3.0, 0.5, 2.5, -1.0])
picks = [top_k_sample(logits, 2, rng) for _ in range(1000)]
print(sorted(set(picks)))                      # [1, 3]
```

핵심

① np.sort(...)[-k]: 오름차순 정렬의 뒤에서 k번째 = 상위 k번째 점수.

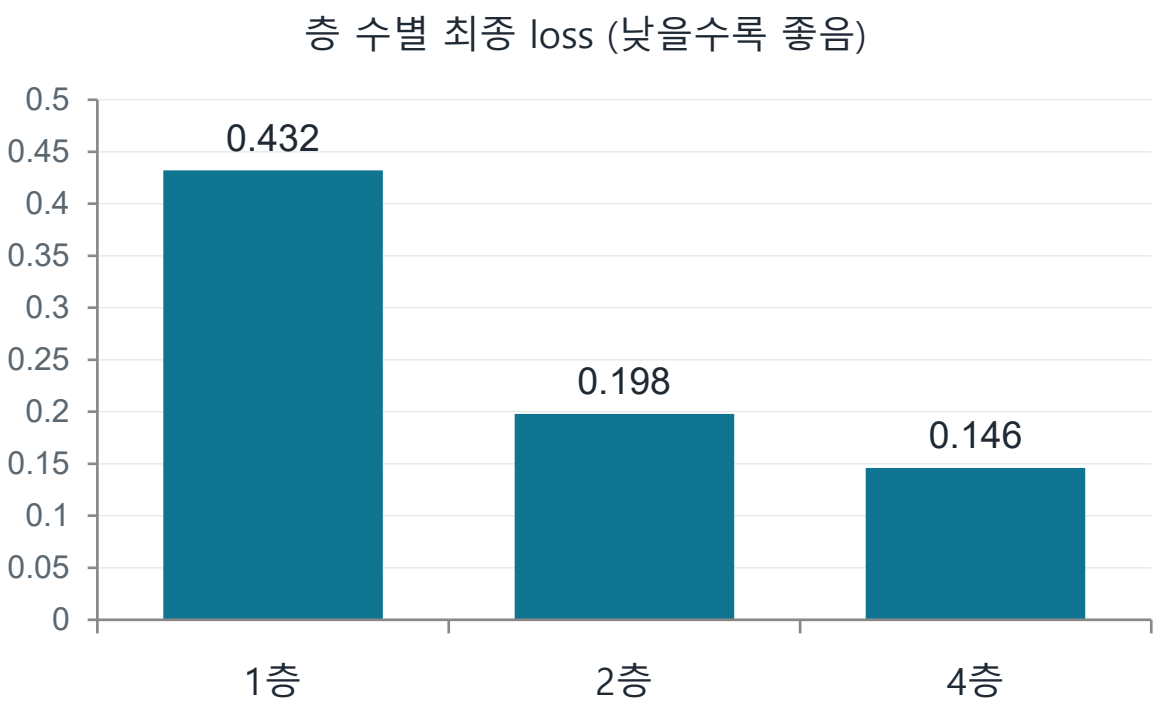
② 그보다 작은 점수는 −inf 로 덮어 탈락시킨다.

③ softmax(−inf) = 0 이므로 탈락자는 절대 뽑히지 않는다.

결과: 1000번 모두 1번(3.0)과 3번(2.5)만 나온다. k=1 이면 항상 최고점수만 뽑는 "greedy" 방식이 된다.

SOLUTION

솔루션 4. 층 수 실험 결과




```
for n_layers in (1, 2, 4):
    model = GPT(V, d_model=64, n_heads=4,
                n_layers=n_layers, dropout=0.0)
```

정답

(1) 1층 0.432 / 2층 0.198 / 4층 0.146
(2) 층이 많을수록 loss가 낮아졌다.
(3) 층이 깊을수록 더 복잡한 패턴을 표현할 수 있다.

주의

- 학습 데이터에 대한 loss일 뿐(검증 loss 아님)
- 너무 키우면 외우기(과적합)만 늘 수 있다.
- 실행 환경에 따라 값은 조금 달라진다.

NEXT STEP

더 공부하려면

도전 과제

- 학습 글을 한국어로 바꿔 보기 (tiny_corpus.txt)
- 위치 인코딩을 학습형 임베딩으로 교체해 비교
- 검증 데이터를 따로 두고 과적합 확인
- BPE 토크나이저 직접 만들기
- KV 캐시로 생성 속도 높이기
- 인코더-디코더(번역) 구조 구현하기

참고 자료

- 논문: Attention Is All You Need (2017)
- Andrej Karpathy: "Let's build GPT" 강의, nanoGPT
- Jay Alammar: The Illustrated Transformer
- PyTorch 공식 문서: torch.nn

오늘 코드: Slider/transformer/
numpy_version/ · pytorch_version/
practice_solutions.py